

# EC200A-CN(TA) QuecOpen

## 分区调整指导

**LTE Standard 模块系列**

版本：1.0.0

日期：2023-03-20

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司  
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233  
电话：+86 21 5108 6236 邮箱：[info@quectel.com](mailto:info@quectel.com)

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：  
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：[support@quectel.com](mailto:support@quectel.com)。

## 前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

## 使用和披露限制

### 许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

### 版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

### 商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

### 第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

## 隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

## 免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

**Copyright © Quectel Wireless Solutions Co., Ltd. 2023.**

# 文档历史

## 修订记录

| 版本    | 日期         | 作者         | 变更表述 |
|-------|------------|------------|------|
| -     | 2022-04-15 | Jessie LEI | 文档创建 |
| 1.0.0 | 2023-03-20 | Nick LIN   | 临时版本 |

## 目录

|                                      |           |
|--------------------------------------|-----------|
| 文档历史 .....                           | 3         |
| 目录 .....                             | 4         |
| 表格索引 .....                           | 5         |
| <b>1 引言 .....</b>                    | <b>6</b>  |
| <b>2 分区调整说明.....</b>                 | <b>7</b>  |
| 2.1. 分区介绍 .....                      | 7         |
| 2.2. 分区调整注意事项 .....                  | 8         |
| <b>3 分区调整方法.....</b>                 | <b>9</b>  |
| 3.1. 调整 Flash Layout .....           | 10        |
| 3.1.1. 涉及文件 .....                    | 10        |
| 3.1.2. 调整方法 .....                    | 10        |
| 3.2. 适配 uboot 配置文件 .....             | 13        |
| 3.2.1. 涉及文件 .....                    | 13        |
| 3.2.2. 适配方法 .....                    | 13        |
| 3.3. 配置 blf 文件 .....                 | 15        |
| 3.3.1. 涉及文件 .....                    | 15        |
| 3.3.2. 配置方法 .....                    | 16        |
| 3.4. 烧录固件 .....                      | 19        |
| 3.5. 添加挂载逻辑到新增分区 .....               | 21        |
| <b>4 UBI 文件系统制作和加载 .....</b>         | <b>24</b> |
| 4.1. 制作 squashfs 文件系统镜像 .....        | 24        |
| 4.2. 加载 squashfs 文件系统 .....          | 25        |
| 4.3. 新增分区 backup_data.ubi 固件制作 ..... | 25        |
| <b>5 附录 参考文档及术语缩写 .....</b>          | <b>27</b> |

## 表格索引

|                                |    |
|--------------------------------|----|
| 表 1: 可调整的 Linux 分区概览.....      | 7  |
| 表 2: update.blf 配置文件标签描述 ..... | 10 |
| 表 3: blf 文件列表 .....            | 15 |
| 表 4: 参考文档 .....                | 27 |
| 表 5: 术语缩写 .....                | 27 |

Preliminary Document  
Not Checked

# 1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen®方案。QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档主要介绍在 QuecOpen®方案下，移远通信 EC200A-CN(TA)模块的分区调整方法以及相关注意事项。

Preliminary Document  
Not Checked

## 2 分区调整说明

### 2.1. 分区介绍

QuecOpen®方案下，Linux 系统中用于存放和读写用户应用程序以及配置参数数据的相关分区如下表所示。

表 1: 可调整的 Linux 分区概览

| 用户分区        | 默认空间   | 可用空间     | 分区格式           | 挂载点             | 分区用途                                        |
|-------------|--------|----------|----------------|-----------------|---------------------------------------------|
| oem_data    | 20 MB  | 约 20 MB  | squashfs 文件系统  | /NVM/oem_data   | 带有 A/B 分区互为备份，支持 OTA 升级，用于存放客户的应用程序和配置参数数据。 |
| rootfs_data | 112 MB | 约 112 MB | ubifs 文件系统     | /data、/mnt、/log | 系统可读写分区                                     |
|             |        |          |                | /overlay/etc    |                                             |
|             |        |          | overlayfs 文件系统 | /etc            |                                             |
|             |        |          |                | /NVM            |                                             |

#### 备注

1. oem\_data-a 和 oem-data-b 分区用于存放用户程序及配置文件，挂载为只读。oem\_data 分区用作 A/B 分区，有备份还原功能，某个分区损坏后会切换系统进行还原恢复；也有 OTA 升级功能。因此建议使用该分区存放重要应用程序及配置文件。
2. rootfs\_data 分区以可读写形式挂载，分为三个 UBI 卷，其中 0 号卷（ubi0\_0）分别挂载到/data、/mnt 和/log 目录，另两个卷分别挂载到/etc 和/NVM 目录。ubi0\_0 卷裸分区大小约 100 MB，可用大小约 81 MB，其中移远通信会存放少量配置数据（≤ 6 MB）。由于分区挂载失败时会擦除该分区/etc 和 /NVM 卷，所以不要直接使用该分区，需新增 backup\_data 分区。新增分区的裸分区大小不大于 89 MB，可用大小不大于 70 MB。若需新增可读写的 backup\_data 分区，请参考第 3 章。

## 2.2. 分区调整注意事项

1. 除调整 `oem_data` 和 `rootfs_data` 分区的大小及新增分区（如可读写的 `backup_data` 分区）外，请勿修改其他分区。
2. 修改 `oem_data` 或 `rootfs_data` 分区大小：
  - `oem_data` 分区挂载在 `/NVM/oemdata` 目录下，建议用户将应用程序存放在该分区以便管理。同时，`oem_data` 分区分为 A 和 B 分区，修改 `oem_data-a` 分区大小时，需同步修改 `oem_data-b` 分区。
  - `rootfs_data` 分区位于 `flash layout` 的最后位置（`QUEEC_BB` 分区为系统预留的分区，不参与分区 `flash layout` 的划分），去除其他分区的占用空间之后，`flash` 的剩余空间都将分给 `rootfs_data`，因此修改 `oem_data-a/oem_data-b` 分区大小后，无需修改 `rootfs_data` 分区大小，只需调整 `rootfs_data` 分区偏移地址即可，但需保证 `rootfs_data` 分区能够正常挂载。

3. 新增分区：

若新增一个可读写的 `backup_data` 分区，建议添加在 `rootfs_data` 分区之前。

## 3 分区调整方法

配置文件 *update.blf* 中定义了镜像列表，和 SWD Downloader 烧录界面中的镜像列表相对应。每个镜像包含了相应的镜像名称、ID 等属性。相关数据段如下所示：

```
[Image_List]
1_Image_Enable = 1
1_Image_Tim_Included = 1
1_Image_Image_ID = 0x54494D48
1_Image_Next_Image_ID = 0x4F424D49
1_Image_Path = tim_falcon_qspinand.bin
.....
12_Image_Flash_Entry_Address = 0x04180000
12_Image_Load_Address = 0xFFFFFFFF
12_Image_Type = RAW
12_Image_ID_Name = GRBI
12_Image_Erase_Size =
12_Image_Partition_Number = 0
12_Image_Hash_Algorithm_ID = SHA-256
12_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
13_Image_Enable = 1
13_Image_Tim_Included = 3
13_Image_Image_ID = 0x52464249
13_Image_Next_Image_ID = 0x4F534C4F
13_Image_Path = RFPLUGIN.bin
13_Image_Flash_Entry_Address = 0x04140000
13_Image_Load_Address = 0xFFFFFFFF
13_Image_Type = RAW
13_Image_ID_Name = RFBI
13_Image_Erase_Size =
13_Image_Partition_Number = 0
13_Image_Hash_Algorithm_ID = SHA-256
13_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
```

表 2: update.blf 配置文件标签描述

| 标签名称                | 描述                                          |
|---------------------|---------------------------------------------|
| TIM Included        | 若设置为 Tim Included，指定镜像将被 Bootloader 自动加载至内存 |
| Flash Entry Address | 指定镜像烧录到 Flash 中的物理地址                        |
| Load Address        | 指定镜像烧录到 DDR Memory 中对应的物理地址                 |

## 3.1. 调整 Flash Layout

### 3.1.1. 涉及文件

*linux/arch/arm/boot/dts/asr1803\_ab\_flash\_layout.dtsi*

### 3.1.2. 调整方法

在相应文件中调整分区设备树 flash layout，如下所示：

```

partition@0 {
    label = "bootloader";
    reg = <0x0 0x40000>;
    read-only;
};

partition@40000 {
    label = "cp_reliabledata";
    reg = <0x40000 0x20000>;
};

. . . . .

partition@100000 {
    label = "dtim-a";
    reg = <0x100000 0x40000>;
};

partition@140000 {
    label = "cpimage-a";
    reg = <0x140000 0xf00000>;
};

partition@1040000 {
    label = "u-boot-a";
    reg = <0x1040000 0xc0000>
};
    
```

```

partition@1100000 {
    label = "kernel-a";
    reg = <0x1100000 0x800000>;
};
partition@1900000 {
    label = "rootfs-a";
    reg = <0x1900000 0x1380000>;
};
partition@2c80000 {
    label = "tos-a";
    reg = <0x2c80000 0x80000>;
};
partition@2d00000 {
    label = "oem_data-a";
    reg = <0x2d00000 0x1400000>;
};
partition@4100000 {
    label = "dtim-b";
    reg = <0x4100000 0x40000>;
};
partition@4140000 {
    label = "cpimage-b";
    reg = <0x4140000 0xf00000>;
};
partition@5040000 {
    label = "u-boot-b";
    reg = <0x5040000 0xc0000>;
};
partition@5100000 {
    label = "kernel-b";
    reg = <0x5100000 0x800000>;
};
partition@5900000 {
    label = "rootfs-b";
    reg = <0x5900000 0x1380000>;
};
partition@6c80000 {
    label = "tos-b";
    reg = <0x6c80000 0x80000>;
};
partition@6d00000 {
    label = "oem_data-b";
    reg = <0x6d00000 0x1400000>;
};

```

```
partition@8100000 {
    label = "asr_flag";
    reg = <0x8100000 0x100000>;
};
partition@8200000 {
    label = "misc";
    reg = <0x8200000 0x40000>;
};
partition@8240000 {
    label = "cust_info";
    reg = <0x8240000 0x40000>;
}; //若需新增分区，推荐在此处添加，并相应调整 rootfs_data 分区
partition@8280000 {
    label = "rootfs_data";
    reg = <0x8280000 0>;
};
```

- 新增分区

以新增一个大小为 16 MB 的可读写分区 backup\_data 为例：

```
partition@8280000 {
    label = "backup_data";
    reg = <0x8280000 0x1000000>;//起止地址
};
partition@9280000 {
    label = "rootfs_data";
    reg = <0x9280000 0>;
}
```

- oem\_data 分区大小调整

以将 oem\_data-a 分区的大小由 20 MB 增加到 30 MB 为例：

入口地址不变，分区大小由 0x1400000 增加到 0x1e00000，同时相应调整 oem\_data-b；由于 oem\_data-a 及 oem\_data-b 分区的调整会导致后面其他分区的入口地址的变化，用户需进行相应调整。

调整前：

```
partition@2d00000 {
    label = "oem_data-a";
    reg = <0x2d00000 0x1400000>;
};
partition@4100000 {
```

```
label = "dtim-b";
reg = <0x4100000 0x40000>;
}
```

调整后:

```
partition@2d00000 {
    label = "oem_data-a";
    reg = <0x2d00000 0x1e00000>;
};
partition@4b00000 {
    label = "dtim-b";
    reg = <0x4b00000 0x40000>;
}
```

## 备注

调整完成后，需重新编译生成 **zImage** 镜像并替换到固件包中；编译方法参考文档 [1]。

## 3.2. 适配 uboot 配置文件

### 3.2.1. 涉及文件

uboot 配置文件: *uboot/include/configs/falcon\_p401.h*

### 3.2.2. 适配方法

在 *uboot/include/configs/falcon\_p401.h* 文件中适配分区表地址参数及 MTD 设备号，如下所示:

```
#define NVM_MTD_PART "rootfs_data"
#define NVM_UBI_VOLUME "nvm"
#define NVM_UBIFS_DIR "/root"
// #define CONFIG_USE_MEP_IN_CODE
#define MEP_OTA_FLASH_ADDRESS 0x000C0000
#define MEP_OTA_FLASH_LEN (0x20000*2)

#define OEM_NVM_MTD_PART "oem_data"
#define OEM_NVM_MTD_PART_A "oem_data-a"
#define OEM_NVM_MTD_PART_B "oem_data-b"

#define OEM_NVM_UBI_VOLUME "oem_data"
#define OEM_NVM_UBIFS_DIR "/config"

#define UBOOT_MTD_PART_A "u-boot-a"
#define UBOOT_MTD_PART_B "u-boot-b"

#define KERNEL_MTD_PART_A "kernel-a"
#define KERNEL_MTD_PART_B "kernel-b"

#define ROOTFS_MTD_PART_A "rootfs-a"
#define ROOTFS_MTD_PART_B "rootfs-b"

#define TOS_MTD_PART_A "tos-a"
#define TOS_MTD_PART_B "tos-b"

#ifdef CONFIG_CMD_FASTBOOT
#define CONFIG_MTDPARTS \
    "mtdparts=nand_mtd:0x00C00000@x1040000("UBOOT_MTD_PART_A"),"\
    "0x00C00000@x5040000("UBOOT_MTD_PART_B"),"\
    "0x08000000@x1100000("KERNEL_MTD_PART_A"),"\
    "0x08000000@x5100000("KERNEL_MTD_PART_B"),"\
    "0x01380000@x1900000("ROOTFS_MTD_PART_A"),"\
    "0x01380000@x5900000("ROOTFS_MTD_PART_B"),"\
    "0x00800000@x2c80000("TOS_MTD_PART_A"),"\
    "0x00800000@x6c80000("TOS_MTD_PART_B"),"\
    "0x14000000@x2d00000("OEM_NVM_MTD_PART_A"),"\
    "0x14000000@x6d00000("OEM_NVM_MTD_PART_B"),"\
    "0x70600000@x8280000("NVM_MTD_PART"),"
#else
```

大小发生改变，以改变后的地址大小填入

调整后rootfs\_data分区的起始地址，发生改变

原 rootfs\_data 分区大小为 0x7060000，起始地址为 0x8280000。由于添加了 backup\_data 分区后，rootfs\_data 分区的大小减去了 backup\_data 分区的大小，故 rootfs\_data 分区大小变为 0x7060000 - 0x1000000 = 0x6060000，起始地址由 0x8280000 变为 0x9280000。

此外，还需同步更新如下参数：

```
/* rootfs data */
#define NVM_MTD_PART_SIZE 0x7060000 /* NOTE: if you change BBM size, you must modify this also */
#define NVM_MTD_PART_ADDR 0x8280000
```

```
#else
#define CONFIG_MTDPARTS \
    "mtdparts=nand_mtd:0x7060000@x8280000("NVM_MTD_PART"),"\
    "0x01400000@x2d00000("OEM_NVM_UBI_VOLUME")"
#endif
```

修改为调整后的地址

```
#ifdef CONFIG_CMD_FASTBOOT
#define CONFIG_MTDPARTS \
    "mtdparts=nand_mtd:0x00C0000@0x1040000("UBOOT_MTD_PART_A"),"\
    "0x00C0000@0x5040000("UBOOT_MTD_PART_B"),"\
    "0x0800000@0x1100000("KERNEL_MTD_PART_A"),"\
    "0x0800000@0x5100000("KERNEL_MTD_PART_B"),"\
    "0x01380000@0x1900000("ROOTFS_MTD_PART_A"),"\
    "0x01380000@0x5900000("ROOTFS_MTD_PART_B"),"\
    "0x00080000@0x2c80000("TOS_MTD_PART_A"),"\
    "0x00080000@0x6c80000("TOS_MTD_PART_B"),"\
    "0x1400000@0x2d00000("OEM_NVM_MTD_PART_A"),"\
    "0x1400000@0x6d00000("OEM_NVM_MTD_PART_B"),"\
    "0x7060000@0x8280000("NVM_MTD_PART"),"
#endif
```

## 备注

1. 适配完成后，需重新编译生成 *u-boot.bin* 镜像并替换到固件包中；编译方法参考文档 [1]。
2. 若 *oem\_data* 分区大小发生调整，上述文件中涉及的分区入口地址及大小需根据 *flash layout* 调整进行相应调整。

## 3.3. 配置 blf 文件

### 3.3.1. 涉及文件

表 3：blf 文件列表

| 文件名称                                         | 描述                  |
|----------------------------------------------|---------------------|
| <i>update.blf</i>                            | SWD Downloader 配置文件 |
| <i>quectel_AB_OTA.blf</i>                    | OTA 配置 blf 文件       |
| <i>quectel_skylark_pm802_standard_AB.blf</i> | 非工厂配置 blf 文件        |

blf 文件是与下载工具 SWD Downloader 配套使用的配置文件。其主要功能如下：

- 配置 DDR 参数
- 配置镜像列表
- 定义其他可配置参数

从使用角度，用户只需关注镜像列表的配置。

### 3.3.2. 配置方法

以调整 oem\_data 分区及新增 backup\_data 分区为例，修改 *update.blf* 文件：

#### 1. 直接修改 *update.blf*：

- 新增 backup\_data 分区供挂载使用

若新增分区不烧录镜像，则无需新增 *image\_list*，调整 backup\_data 分区后的 rootfs\_data 分区大小及入口地址即可。

```

End_VIA_COMBINED_INFO_TRANSFER_TORF_ID
End_Reserved_Data
[EraseOnly_Option]
Total_Eraseonly_Areas = 1          rootfs_data分区
1_Eraseonly_Area_Size = 0x07060000
1_Eraseonly_Area_FlashStartAddress = 0x08280000
1_Eraseonly_Area_Partition = 0
1_Eraseonly_Area_Enable = 1        rootfs_data入口地址
[Extended_Reserved_Data]
Consumer_ID
CID = TBRI
    
```

若新增分区需烧录镜像，则需新增 *image\_list*。

```

End_VIA_COMBINED_INFO_TRANSFER_TORF_ID
End_Reserved_Data
[EraseOnly_Option]
Total_Eraseonly_Areas = 1          rootfs_data分区
1_Eraseonly_Area_Size = 0x07060000
1_Eraseonly_Area_FlashStartAddress = 0x08280000
1_Eraseonly_Area_Partition = 0
1_Eraseonly_Area_Enable = 1        rootfs_data入口地址
[Extended_Reserved_Data]
Consumer_ID
CID = TBRI
    
```

示例如下：

#### a. 修改镜像数

Number\_of\_Images = 18 改成 Number\_of\_Images = 19

#### b. 修改 *image\_list*

```

18_Image_Enable = 1
18_Image_Tim_Included = 3
18_Image_Image_ID = 0x4F454D44
18_Image_Next_Image_ID = 0x44415441 //由 0xFFFFFFFF 改为 0x44415441
18_Image_Path = oem_data.ubi
18_Image_Flash_Entry_Address = 0x06D00000
18_Image_Load_Address = 0xFFFFFFFF
18_Image_Type = RAW
18_Image_ID_Name = OEMD
    
```

```

18_Image_Erase_Size = 0x01400000
18_Image_Partition_Number = 0
18_Image_Hash_Algorithm_ID = SHA-256
18_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
//////////新增下面这个 image list
19_Image_Enable = 1
19_Image_Tim_Included =
19_Image_Image_ID = 0x44415441
19_Image_Next_Image_ID = 0xFFFFFFFF
19_Image_Path = backup_data.ubi
19_Image_Flash_Entry_Address = ////////////backup_data 分区的入口地址
19_Image_Load_Address = 0xFFFFFFFF
19_Image_Type = RAW
19_Image_ID_Name = DATA
19_Image_Erase_Size = ////////////backup_data 分区的大小
19_Image_Partition_Number = 0
19_Image_Hash_Algorithm_ID = SHA-256
19_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF

```

- 调整 oem\_data 分区

若调整了 oem\_data-a、oem\_data-b 分区的大小，需修改 *update.blf* 文件相关分区配置。由于 flash layout 中 asr\_flag 分区位于 oem\_data 分区之后，首先需调整非烧录分区 asr\_flag 分区，如下所示：

```

OTAI
Enabled = 0x00000001
Flash_Address = 0x08100000
Magic = 0x464F5441
End_OTAI
BBMT

```

asr\_flag分区入口  
地址调整

其次，dtim-b 分区的入口地址发生相应偏移，也需修改：

```

3_Image_Map_Info
3_Image_ID = 0x54494D32
3_Image_Type = RECOVERYIMAGE
3_Flash_Address_Lo = 0x04100000
3_Flash_Address_Hi = 0x00000000
3_Partition = 0x00000000
3_Enable = 1
3_End_Image_Map_Info
End_Image_Maps
Vendor_DDR_Initialization

```

dtim-b分区镜像类型  
dtim-b分区入口地址

此外，还需修改 oem\_data 分区后有镜像的分区的入口地址：

```

10_Image_Enable = 1
10_Image_Tim_Included = 2
10_Image_Image_ID = 0x4F454D44 oem_data分区固件
10_Image_Next_Image_ID = 0x41524249
10_Image_Path = oem_data.ubi
10_Image_Flash_Entry_Address = 0x02D00000 oem_data分区入口地址
10_Image_Load_Address = 0xFFFFFFFF
10_Image_Type = RAW
10_Image_ID_Name = OEMD
10_Image_Erase_Size = 0x01400000
10_Image_Partition_Number = 0
10_Image_Hash_Algorithm_ID = SHA-256
10_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
11_Image_Enable = 1
11_Image_Tim_Included = 3
11_Image_Image_ID = 0x41524249
11_Image_Next_Image_ID = 0x47524249
11_Image_Path = ARBEL.bin
11_Image_Flash_Entry_Address = 0x04580000
11_Image_Load_Address = 0xFFFFFFFF
11_Image_Type = RAW
11_Image_ID_Name = ARBI
11_Image_Erase_Size =
11_Image_Partition_Number = 0
11_Image_Hash_Algorithm_ID = SHA-256
11_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
12_Image_Enable = 1
12_Image_Tim_Included = 3
12_Image_Image_ID = 0x47524249
12_Image_Next_Image_ID = 0x52464249
12_Image_Path = MSA.bin
12_Image_Flash_Entry_Address = 0x04180000
12_Image_Load_Address = 0xFFFFFFFF
12_Image_Type = RAW
12_Image_ID_Name = GRBI
12_Image_Erase_Size =
12_Image_Partition_Number = 0
12_Image_Hash_Algorithm_ID = SHA-256
12_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
13_Image_Enable = 1
13_Image_Tim_Included = 3
13_Image_Image_ID = 0x52464249
13_Image_Next_Image_ID = 0x4F534C4F
13_Image_Path = RFFLOGIN.bin
13_Image_Flash_Entry_Address = 0x04140000
13_Image_Load_Address = 0xFFFFFFFF
13_Image_Type = RAW
13_Image_ID_Name = RFBI
13_Image_Erase_Size =
13_Image_Partition_Number = 0
13_Image_Hash_Algorithm_ID = SHA-256
13_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
14_Image_Enable = 1
14_Image_Tim_Included = 3
14_Image_Image_ID = 0x4F534C4F
14_Image_Next_Image_ID = 0x5A494D47
14_Image_Path = u-boot.bin
14_Image_Flash_Entry_Address = 0x05040000
14_Image_Load_Address = 0x05080000
14_Image_Type = RAW
14_Image_ID_Name = OSLO
14_Image_Erase_Size =
14_Image_Partition_Number = 0
14_Image_Hash_Algorithm_ID = SHA-256
14_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
15_Image_Enable = 1
15_Image_Tim_Included = 3
15_Image_Image_ID = 0x5A494D47
15_Image_Next_Image_ID = 0x5359534A
15_Image_Path = zImage
15_Image_Flash_Entry_Address = 0x05100000
15_Image_Load_Address = 0xFFFFFFFF
15_Image_Type = RAW
15_Image_ID_Name = ZIMG
15_Image_Erase_Size =
15_Image_Partition_Number = 0
15_Image_Hash_Algorithm_ID = SHA-256
15_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
16_Image_Enable = 1
16_Image_Tim_Included = 3
16_Image_Image_ID = 0x5359534A
16_Image_Next_Image_ID = 0x545A5349
16_Image_Path = root.squashfs
16_Image_Flash_Entry_Address = 0x05900000
16_Image_Load_Address = 0xFFFFFFFF
16_Image_Type = RAW
16_Image_ID_Name = SYSJ
16_Image_Erase_Size = 0x01380000
16_Image_Partition_Number = 0
16_Image_Hash_Algorithm_ID = SHA-256
16_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
17_Image_Enable = 1
17_Image_Tim_Included = 3
17_Image_Image_ID = 0x545A5349
17_Image_Next_Image_ID = 0x4F454D44
17_Image_Path = tos.bin
17_Image_Flash_Entry_Address = 0x06C80000
17_Image_Load_Address = 0x02000000
17_Image_Type = RAW
17_Image_ID_Name = TZSI
17_Image_Erase_Size =
17_Image_Partition_Number = 0
17_Image_Hash_Algorithm_ID = SHA-256
17_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
18_Image_Enable = 1
18_Image_Tim_Included = 3
18_Image_Image_ID = 0x4F454D44
18_Image_Next_Image_ID = 0xFFFFFFFF
18_Image_Path = oem_data.ubi
18_Image_Flash_Entry_Address = 0x06D00000
18_Image_Load_Address = 0xFFFFFFFF
18_Image_Type = RAW
18_Image_ID_Name = OEMD
18_Image_Erase_Size = 0x01400000
18_Image_Partition_Number = 0
18_Image_Hash_Algorithm_ID = SHA-256

```

随着oem\_data分区的偏移进行偏移

2. 使用 `part_resizing.sh` 脚本辅助修改 blf 文件，修改后的 blf 文件位于 `tools/fota/target` 路径：

- 新增 backup\_data 分区大小为 16 MB：

```
./part_resizing.sh backup_data 16
```

```
ol@ol:~/work/code/asr/quectel_dev_sec_latest/sec_1122/ql-ol-extsdk-ec200acntar02a06n2gv01/tools/fota$ ./part_resizing.sh backup_data 16
NEW_DATA_NAME: backup_data
NEW_DATA_SIZE: 16 M
NEW_OEM_DATA_SIZE: 20 M
rootfs_data size:0x06060000
rootfs_data address:0x09280000
ol@ol:~/work/code/asr/quectel_dev_sec_latest/sec_1122/ql-ol-extsdk-ec200acntar02a06n2gv01/tools/fota$ ll target/
total 128
drwxrwxr-x 2 ol ol 4096 11月 22 17:50 ./
drwxrwxr-x 3 ol ol 4096 11月 22 16:52 ../
-rw-r--r-- 1 ol ol 15315 11月 22 17:50 quectel_AB_OTA.blf
-rw-rw-r-- 1 ol ol 52394 11月 22 17:50 quectel_skylark_pm802_standard_AB.blf
-rw-r--r-- 1 ol ol 52814 11月 22 17:50 update.blf
ol@ol:~/work/code/asr/quectel_dev_sec_latest/sec_1122/ql-ol-extsdk-ec200acntar02a06n2gv01/tools/fota$
```

- 新增 backup\_data 分区大小为 16 MB，并调整 oem\_data 分区大小为 30 MB：

```
./part_resizing.sh backup_data 16 30
```

```
ol@ol:~/work/code/asr/quectel_dev_sec_latest/sec_1122/ql-ol-extsdk-ec200acntar02a06n2gv01/tools/fota$ ./part_resizing.sh backup_data 16 30
NEW_DATA_NAME: backup_data
NEW_DATA_SIZE: 16 M
NEW_OEM_DATA_SIZE: 30 M
rootfs_data size:0x04C60000
rootfs_data address:0x0A680000
oem-data size:0x01E00000
dtim-b address:0x04B00000
RFPLUGIN address:0x04B40000
MSA address:0x04B80000
ARBEL address:0x04F80000
uboot-b address:0x05A40000
kernel-b address:0x05B00000
rootfs-b address:0x06300000
tos-b address:0x07680000
oem-data-b address:0x07700000
asr_flag address:0x09500000
ol@ol:~/work/code/asr/quectel_dev_sec_latest/sec_1122/ql-ol-extsdk-ec200acntar02a06n2gv01/tools/fota$ ll target/
total 128
drwxrwxr-x 2 ol ol 4096 11月 22 17:52 ./
drwxrwxr-x 3 ol ol 4096 11月 22 16:52 ../
-rw-r--r-- 1 ol ol 15304 11月 22 17:52 quectel_AB_OTA.blf
-rw-rw-r-- 1 ol ol 52382 11月 22 17:52 quectel_skylark_pm802_standard_AB.blf
-rw-r--r-- 1 ol ol 52802 11月 22 17:52 update.blf
ol@ol:~/work/code/asr/quectel_dev_sec_latest/sec_1122/ql-ol-extsdk-ec200acntar02a06n2gv01/tools/fota$
```

## 备注

如需 OTA 升级，需修改 `quectel_skylark_pm802_standard_AB.blf` 和 `quectel_AB_OTA.blf` 制作 OTA 升级包，修改方式可参考 `update.blf` 文件的修改，也可使用 `part_resizing.sh` 脚本辅助修改。

## 3.4. 烧录固件

将 `zimage`（详见第 3.1.2 章）、`u-boot.bin`（详见第 3.2.2 章）和 blf 文件（详见第 3.3.2 章）替换到固件包内并烧录至模块后，分区修改生效。执行 `cat /proc/mtd` 显示 MTD 分区信息，如下所示：

```

root@OpenWrt:~# cat /proc/mtd
dev:   size   erasesize  name
mtd0: 00040000 00020000 "bootloader"
mtd1: 00020000 00020000 "cp_reliabledata"
mtd2: 00020000 00020000 "ap_reliabledata"
mtd3: 00020000 00020000 "mep-ota"
mtd4: 00020000 00020000 "cp_reliabledata_backup"
mtd5: 00020000 00020000 "ap_reliabledata_backup"
mtd6: 00020000 00020000 "mep-ota_backup"
mtd7: 00040000 00020000 "dtim-a"
mtd8: 00f00000 00020000 "cpimage-a"
mtd9: 000c0000 00020000 "u-boot-a"
mtd10: 00800000 00020000 "kernel-a"
mtd11: 01380000 00020000 "rootfs-a"
mtd12: 00080000 00020000 "tos-a"
mtd13: 01e00000 00020000 "oem_data-a" //分区大小由原来的 20 MB (0x01400000) 增加到现在的
                                         30 MB (0x01e00000)

mtd14: 00040000 00020000 "dtim-b"
mtd15: 00f00000 00020000 "cpimage-b"
mtd16: 000c0000 00020000 "u-boot-b"
mtd17: 00800000 00020000 "kernel-b"
mtd18: 01380000 00020000 "rootfs-b"
mtd19: 00080000 00020000 "tos-b"
mtd20: 01e00000 00020000 "oem_data-b" //分区大小由原来的 20 MB (0x01400000) 增加到现在的
                                         30 MB (0x01e00000)

mtd21: 00100000 00020000 "asr_flag"
mtd22: 00040000 00020000 "misc"
mtd23: 00040000 00020000 "cust_info"
mtd24: 01000000 00020000 "backup_data" //新增 backup_data 分区大小为 16 MB (0x01000000)
mtd25: 04c60000 00020000 "rootfs_data" //分区大小由原来的 112 MB (0x07060000) 减小到现在的
                                         76 MB (0x04c60000), 减少的 36 MB 一部分用于新增 data
                                         分区 (16 MB), 另外一部分用于 oem_data-a 扩容 (10 MB)
                                         及 oem_data-b 扩容 (10 MB)。

mtd26: 00d20000 00020000 "QUEC_BBM"

```

## 备注

由于调整分区对其后分区会产生影响并需相应修改，非必要不建议调整分区表。

### 3.5. 添加挂载逻辑到新增分区

需要挂载使用的分区，可以使用 `ql-ol-rootfs/lib/preinit/81_mount_ubifs_overlay` 脚本对其进行调整。以将新增 `backup_data` 分区挂载为 `ubifs` 文件系统为例，添加挂载逻辑如下：

#### 1. 获取新增分区的 MTD 号：

```
ubifs_volume_support()
    mtdpart_idx="$(find_mtd_index rootfs_data)"
    [ -z "$mtdpart_idx" ] && return 1
    + mtdpart_idx_backup_data="$(find_mtd_index backup_data)"
    + [ -z "$mtdpart_idx_backup_data" ] && return 1
    mtdpart_idx_oem="$(find_mtd_index oem_data$SLOT)"
    if [ -z "$mtdpart_idx_oem" ]
    then
        # oem_data may has only one partition
        mtdpart_idx_oem="$(find_mtd_index oem_data)"
        [ -z "$mtdpart_idx_oem" ] && return 1
    fi
    grep -qs ubifs /proc/filesystems || return 1
    echo "found rootfs_data partition and ubifs support"
    return 0
}
```

#### 2. 添加挂载 ubifs 的可读写挂载目录 `/mnt/backup_data`：

```
ubifs_backup_data_mount() {
    recover_ubifs=0          ///recover_ubifs 为 0，挂载正常；为 1，检查挂载所需设备节点异常，格式化重新创建节点；
    [ ! -e /dev/ubi2 ] && ubiattach /dev/ubi_ctrl -m $mtdpart_idx_backup_data -d 2 ||
    recover_ubifs=1 //ubi 设备异常
    if [ $recover_ubifs -eq 0 ]
    then
        ubi2_nod_id=`cat /sys/class/ubi/ubi2/dev | tr -s ":" " "`
        [ ! -e /dev/ubi2 ] && mknod /dev/ubi2 c ${ubi2_nod_id}
        if [ ! -e /sys/class/ubi/ubi2_0/dev ]
        then
            # no volume
            recover_ubifs=1          //卷设备异常
        else
            # check for "backup_data" volume
            ubi2_0_nod_id=`cat /sys/class/ubi/ubi2_0/dev | tr -s ":" " "`
            [ ! -e /dev/ubi2_0 ] && mknod /dev/ubi2_0 c ${ubi2_0_nod_id}
            { ubinfo /dev/ubi2_0 | grep Name | grep -qs "backup_data" ; } || \
```

```

        recover_ubifs=1      //卷设备节点异常
    fi
fi
if [ $recover_ubifs -eq 1 ]    ///检查设备及节点异常后进行分区格式化重新创建修复
then
    echo "ubifs data partition is damaged"
    echo "try to recover by formatting $mtdpart..."
    [ -e /dev/ubi2 ] && ubidetach -m $mtdpart_idx_backup_data
    ubiformat -y -q /dev/mtd$mtdpart_idx_backup_data    //格式化 mtd
    ubiattach -m $mtdpart_idx_backup_data /dev/ubi_ctrl    //做 ubi 设备
    ubi2_nod_id=`cat /sys/class/ubi/ubi2/dev | tr -s ":" " "`
    [ ! -e /dev/ubi2 ] && mknod /dev/ubi2 c ${ubi2_nod_id}
    ubimkvol /dev/ubi2 -n 0 -N backup_data -t dynamic --maxavsize    //做卷设备
fi
    mount -t ubifs -o rw,noatime,bulk_read ubi2:backup_data /mnt/backup_data    //挂载
return 0
}

```

### 3. 调用 `ubifs_data_mount`:

```

ubifs_syscfg_rootfs_pivot() {
    echo "switching to ubifs sysfs overlay"
    # PIPE mode need to recover this file in /etc/init.d/network
    cp -rf /etc/config/network /tmp/network_pipe
    # Dir /etc and /NVM can be written after creating overlay
    create_overlay /etc $overlay_mountpoint/etc /etc
    create_overlay /NVM $overlay_mountpoint/nvm /NVM
    #jessie.lei-2022/6/7 During the ota upgrade process, the rootfs_data partition will be erased,
    #and the ota upgrade state will be erased, so the erased NEEDSYNCS state is written here during
    the ota upgrade.
    update_flag=`hexdump -n 8 -s 176 /dev/mtdblock21 | awk '{print$2}'`
    if [ $update_flag -eq 1 ];
    then
        touch /NVM/.fota_state.txt
        echo 5 > /NVM/.fota_state.txt
        #Prevent the status query from failing after the first burning flash
        elif [ ! -e "/NVM/.fota_state.txt" ];
        then
            touch /NVM/.fota_state.txt
            echo 0 > /NVM/.fota_state.txt
        fi
        ubifs_oem_data_mount
        + mkdir -p /mnt/backup_data    //创建挂载点
        + ubifs_backup_data_mount    //调用挂载功能
    fi
}

```

```
}
```

挂载完成后，新增 data 分区的状态如下所示：

```
root@OpenWrt:~# df -h
Filesystem      Size      Used Available Use% Mounted on
/dev/dm-0        14.3M      14.3M          0 100% /
tmpfs            45.5M      112.0K    45.4M   0% /tmp
ubi0:data        53.2M      32.0K    50.4M   0% /data
ubi0:data        53.2M      32.0K    50.4M   0% /mnt
ubi0:data        53.2M      32.0K    50.4M   0% /log
ubi0:etc         2.9M       1.8M    964.0K  65% /overlay/etc
ubi0:nvm         2.0M       56.0K    1.8M    3% /overlay/nvm
overlayfs:/overlay/etc 2.9M       1.8M    964.0K  65% /etc
overlayfs:/overlay/nvm 2.0M       56.0K    1.8M    3% /NVM
/dev/ubiblock1_0 128.0K     128.0K          0 100% /NVM/oem_data
ubi2:backup_data 7.5M       20.0K     7.5M    0% /mnt/backup_data
tmpfs           512.0K          0    512.0K   0% /dev
/dev/mmcblk0p2   1.5G      28.0K    1.4G    0% /mnt/mmcblk0p2
/dev/mmcblk0p1   1.9G     30.3M    1.8G    2% /mnt/mmcblk0p1
root@OpenWrt:~# mount
/dev/dm-0 on / type squashfs (ro,seclabel,relatime)
sysfs on /sys type sysfs (rw,seclabel,relatime)
selinuxfs on /sys/fs/selinux type selinuxfs (rw,nosuid,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,noatime)
tmpfs on /tmp type tmpfs (rw,seclabel,nosuid,nodev,noatime)
ubi0:data on /data type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=0)
ubi0:data on /mnt type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=0)
ubi0:data on /log type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=0)
ubi0:etc on /overlay/etc type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=1)
ubi0:nvm on /overlay/nvm type ubifs (rw,seclabel,noatime,assert=read-only,ubi=0,vol=2)
overlayfs:/overlay/etc on /etc type overlay (rw,seclabel,noatime,lowerdir=/etc,upperdir=/overlay/etc/root,workdir=/overlay/etc/work)
overlayfs:/overlay/nvm on /NVM type overlay (rw,seclabel,noatime,lowerdir=/NVM,upperdir=/overlay/nvm/root,workdir=/overlay/nvm/work)
/dev/ubiblock1_0 on /NVM/oem_data type squashfs (ro,seclabel,relatime)
ubi2:backup_data on /mnt/backup_data type ubifs (rw,seclabel,noatime,bulk_read,assert=read-only,ubi=2,vol=0)
tmpfs on /dev type tmpfs (rw,seclabel,nosuid,relatime,size=512k,modes=755)
devpts on /dev/pts type devpts (rw,seclabel,nosuid,noexec,relatime,modes=600,ptmxmode=000)
/dev/mmcblk0p2 on /mnt/mmcblk0p2 type ext4 (rw,seclabel,nosuid,nodev,relatime)
/dev/mmcblk0p1 on /mnt/mmcblk0p1 type ext4 (rw,seclabel,nosuid,nodev,relatime)
debugfs on /sys/kernel/debug type debugfs (rw,seclabel,noatime)
none on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,noatime,modes=700)
securityfs on /sys/kernel/security type securityfs (rw,relatime)
root@OpenWrt:~#
```

# 4 UBI 固件制作和加载

本章以 EC200ACNTAR02A02M2G\_OCPU 版本为例介绍 UBI 固件的制作和加载方法。

## 4.1. oem\_data.ubi 制作和加载

用户使用 oem\_data 分区将应用程序及配置文件打包进 oem\_data.ubi 固件并烧录至模块。squashfs 文件系统的镜像制作和加载方法如下所示：

### 4.1.1. 制作 squashfs 文件系统镜像

解压 SDK 后进入 /ql-ol-extsdk-ec200acntar02a02m2g\_ocpu/ 目录下，执行 **make oemapp** 创建 /oemapp/ 目录。该目录下有 /config/ 目录用于存放 GKI 相关配置，用户无需关注。将用户数据存入 /oemapp/ 目录下，然后执行 **make oemapp** 在 /target/ 目录下生成 oem\_data.ubi。该 UBI 固件打包了 oem\_data.squashfs。最后将生成的 oem\_data.ubi 新固件替换到固件包内进行烧录，或通过 FOTA 方式进行烧录。

```
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/oemapp$ ls
config
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/oemapp$ mkdir cust_app
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/oemapp$ ls
config  cust_app
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/oemapp$ cd ..
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu$ make oemapp
mkdir -p /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target; \
mkdir -p /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/oemapp; \
mkdir -p /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/oemapp/config; \
fakeroot /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/tools/bin/mksquashfs4 /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/oemapp /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/oem_data.squashfs -nopad -noappend -root-owned -comp gzip -b 128k -processors 1; \
cp /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/tools/conf/ubinize-oem.cfg /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target
Parallel mksquashfs: Using 1 processor
Creating 4.0 filesystem on /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/oem_data.squashfs, block size 131072.

Exportable Squashfs 4.0 filesystem, gzip compressed, data block size 131072
  compressed data, compressed metadata, compressed fragments,
  compressed xattrs, compressed ids
  duplicates are removed
Filesystem size 0.22 Kbytes (0.00 Mbytes)
  78.37% of uncompressed filesystem size (0.28 Kbytes)
Inode table size 46 bytes (0.04 Kbytes)
  46.94% of uncompressed inode table size (98 bytes)
Directory table size 39 bytes (0.04 Kbytes)
  88.64% of uncompressed directory table size (44 bytes)
Number of duplicate files found 0
Number of inodes 3
Number of files 0
Number of fragments 0
Number of symbolic links 0
Number of device nodes 0
Number of fifo nodes 0
Number of socket nodes 0
Number of directories 3
Number of ids (unique uids + gids) 1
Number of uids 1
  root (0)
Number of gids 1
  root (0)
cd /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target; \
fakeroot /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/tools/bin/ubinize -o /home/ol/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/oem_data.ubi -m 2048 -p 128KiB -s 2048 /home/ol
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu$ cd target/
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target$ ls
oem_data.squashfs  oem_data.ubi  ubinize-oem.cfg
ol@ol:~/code/ASR/new/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target$
```

### 4.1.2. 加载 squashfs 文件系统

squashfs 文件系统的加载，第一步是创建设备节点，第二步是 squashfs 文件系统的挂载。在系统重启时执行 `ql-ol-extsdk-ec200acntar02a02m2g_ocpu/ql-ol-rootfs/lib/preinit/81_mount_ubifs_overlay` 脚本即可实现此过程。

## 4.2. backup\_data.ubi 制作

1. 修改 Makefile，新增 backup\_data 分区，如下图所示：

```
export QL_TARGET_DIR=$(QL_SDK_DIR)/target
export IMAGE SYSFS UBI NAME=root.squashfs
export QL_OEMAPP_DIR=$(QL_SDK_DIR)/oemapp
export QL_BACKUP_DIR=$(QL_SDK_DIR)/backup_data
$(shell mkdir -p target)

.PHONY:all

oemapp:
    mkdir -p ${QL_TARGET_DIR}; \
    mkdir -p ${QL_OEMAPP_DIR}; \
    mkdir -p ${QL_OEMAPP_DIR}/config; \
    fakeroot ${QL_TOOLS_BIN_DIR}/mksquashfs4 ${QL_OEMAPP_DIR} ${QL_TARGET_DIR}/oem_data.squashfs -nopad -noappend -root-owned -comp gzip -b 128k -pro
    cessors 1; \
    cp ${QL_TOOLS_CONF_DIR}/ubinize-oem.cfg ${QL_TARGET_DIR}
    cd ${QL_TARGET_DIR}; \
    fakeroot ${QL_TOOLS_BIN_DIR}/ubinize -o ${QL_TARGET_DIR}/oem_data.ubi -m 2048 -p 128KiB -s 2048 ${QL_TARGET_DIR}/ubinize-oem.cfg

backup_data:
    mkdir -p ${QL_TARGET_DIR}; \
    mkdir -p ${QL_BACKUP_DIR}; \
    mkdir -p ${QL_BACKUP_DIR}/config; \
    fakeroot ${QL_TOOLS_BIN_DIR}/mksquashfs4 ${QL_BACKUP_DIR} ${QL_TARGET_DIR}/backup_data.squashfs -nopad -noappend -root-owned -comp gzip -b 128k -
    processors 1; \
    cp ${QL_TOOLS_CONF_DIR}/ubinize-backup.cfg ${QL_TARGET_DIR}
    cd ${QL_TARGET_DIR}; \
    fakeroot ${QL_TOOLS_BIN_DIR}/ubinize -o ${QL_TARGET_DIR}/backup_data.ubi -m 2048 -p 128KiB -s 2048 ${QL_TARGET_DIR}/ubinize-backup.cfg

.PHONY:sample
```

2. 参考 `ubinize-oem.cfg` 在 `ql-ol-extsdk-ec200acntar02a02m2g_ocpu/tools/conf` 下新建 `ubinize-backup.cfg`:

```
[backup_data]
# Volume mode (other option is static)
mode=ubi
# Source image
image=backup_data.squashfs
# Volume ID in UBI image
vol_id=0
# Allow for dynamic resize
vol_type=dynamic
#vol_type=static
# Volume name
vol_name=backup_data
#
# Total amount of logical eraseblocks minus 4(two for internal Volume,
# one for Wear-leveling, one for Scrubbing), then multiplied by 124KB
vol_size=9424KiB
#vol_flags=autoresize
```

其中 vol\_size 计算方法如下:

逻辑擦除块总数 = backup\_data 分区大小/物理块大小;  $\text{vol\_size} = (\text{逻辑擦除块总数} - 4) \times \text{逻辑块大小}$ 。其中, 减去的 4 个块 2 个用于内部卷, 1 个用于磨损平衡, 1 个用于擦洗。

例如, 若 backup\_data 分区大小为 10 MiB, 物理块大小为 128 KiB, 逻辑块大小为 124 KiB, 则  $\text{vol\_size} = [(10 \times 1024)/128 - 4] \times 124 = 9424 \text{ KiB}$ 。

3. 在 `ql-ol-extsdk-ec200acntar02a02m2g_ocpu/` 根目录下执行 **make backup\_data** 生成 `backup_data.ubi`:

```

ql-ol-extsdk-ec200acntar02a02m2g_ocpu$ make backup_data
mkdir -p /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target; \
mkdir -p /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/backup_data; \
mkdir -p /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/backup_data/config; \
fakeroot /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/tools/bin/mksquashfs4 /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/backup_data /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/backup_data.squashfs -nopad -noappend -root-owned -comp gzip -b 128k -processors 1; \
cp /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/tools/conf/ubinize-backup.cfg /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target
Parallel mksquashfs: Using 1 processor
Creating 4.0 filesystem on /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/backup_data.squashfs, block size 131072.

Exportable Squashfs 4.0 filesystem, gzip compressed, data block size 131072
  compressed data, compressed metadata, compressed fragments,
  compressed xattrs, compressed ids
  duplicates are removed
Filesystem size 0.19 Kbytes (0.00 Mbytes)
  87.17% of uncompressed filesystem size (0.22 Kbytes)
Inode table size 39 bytes (0.04 Kbytes)
  59.09% of uncompressed inode table size (66 bytes)
Directory table size 24 bytes (0.02 Kbytes)
  85.71% of uncompressed directory table size (28 bytes)
Number of duplicate files found 0
Number of inodes 2
Number of files 0
Number of fragments 0
Number of symbolic links 0
Number of device nodes 0
Number of fifo nodes 0
Number of socket nodes 0
Number of directories 2
Number of ids (unique uids + gids) 1
Number of uids 1
  root (0)
Number of gids 1
  root (0)
cd /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target; \
fakeroot /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/tools/bin/ubinize -o /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/backup_data.ubi -n 2048 -p 128KiB -s 2048 /home/ol/work2/code/asr/ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/ubinize-backup.cfg

```

生成的 UBI 固件位于 `ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target/`目录下:

```

ql-ol-extsdk-ec200acntar02a02m2g_ocpu/target$ ls
backup_data.squashfs  backup_data.ubi  root.squashfs  root.squashfs.fakeroot-script  ubinize-backup.cfg

```

## 5 附录 参考文档及术语缩写

表 4: 参考文档

| 文档名称                                      |
|-------------------------------------------|
| [1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导 |

表 5: 术语缩写

| 缩写   | 英文全称                     | 中文全称    |
|------|--------------------------|---------|
| DDR  | Double Data Rate         | 双倍数据速率  |
| FOTA | Firmware Over-The-Air    | 固件空中升级  |
| GKI  | General Kernel Image     | 通用内核镜像  |
| ID   | Identifier               | 标识符     |
| IoT  | Internet of Things       | 物联网     |
| MTD  | Memory Technology Device | 存储器技术设备 |
| SDK  | Software Development Kit | 软件开发工具包 |
| UBI  | Unsorted Block Image     | 无排序区块图像 |